

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE CIENCIAS



**ESCUELA PROFESIONAL DE CIENCIA DE LA
COMPUTACIÓN**

**Implementación de un Robot Micromouse para
la Exploración y Resolución de Laberintos**

INTEGRANTES:

	Nombre	Código
Integrante 1	Rydell Jonel Mosquera Huayhua	20232690K
Integrante 2	Danna Ninel Pinto Salas	20234190E
Integrante 3	Brayan Sánchez Zamora	20235010K
Integrante 4	Jeremy Alexander Cosavalente Mendoza	20232189J

NOMBRE DEL DOCENTE:

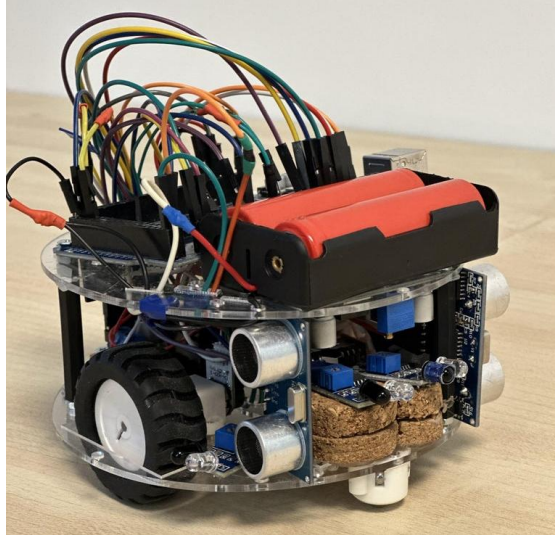
CÉSAR MARTÍN CRUZ SALAZAR

PERIODO: 2025-1

DEFINICIONES DE TERMINOS TECNICOS

1. **Micromouse**

Es un robot móvil autónomo diseñado para recorrer un laberinto desde un punto de inicio hasta un objetivo, tomando decisiones en tiempo real. Su desarrollo implica conocimientos de programación, electrónica y algoritmos de búsqueda de caminos.



Fuente: Instructables, “Micro Mouse for Beginners”, disponible en:
<https://www.instructables.com/Micro-Mouse-for-Beginnersth/>

2. **Microcontrolador**

Dispositivo electrónico programable que integra procesador, memoria y puertos de entrada/salida en un solo chip. Se encarga de ejecutar el software que controla el comportamiento del robot, como el movimiento, la lectura de sensores y la toma de decisiones.

3. **Unidad de control**

Es el bloque lógico encargado de coordinar las operaciones del robot. En el contexto del Micromouse, puede referirse tanto al código que gestiona el flujo de tareas como al propio microcontrolador que ejecuta dicho código.

4. **I/O Mapping (Mapeo de Entradas/Salidas)**

Proceso mediante el cual se definen qué pines del microcontrolador se usarán para leer datos (entradas, como sensores de distancia) o para enviar señales (salidas, como motores). Es fundamental para establecer la comunicación correcta entre el software y el hardware.

5. **Sensor ultrasónico**

Dispositivo que mide distancias utilizando ondas de sonido de alta frecuencia.

Es útil en el Micromouse para detectar paredes y calcular espacios libres en el laberinto.



*Fuente: SparkFun, “Ultrasonic Distance Sensor - HC-SR04”, disponible en:
<https://www.sparkfun.com/products/15569>*

6. **Sensor de efecto Hall**

Sensor que detecta la presencia de campos magnéticos. Puede ser utilizado en encoders magnéticos para medir la rotación de las ruedas, ayudando a estimar la distancia recorrida.

7. **Encoder**

Dispositivo que genera señales digitales basadas en el movimiento de rotación, permitiendo medir la velocidad o el desplazamiento de una rueda. Se utiliza para que el Micromouse mantenga un movimiento preciso y estable.

8. **Memoria EEPROM**

Memoria no volátil interna al microcontrolador. Permite almacenar datos de forma permanente, como parámetros de configuración o rutas exploradas, incluso cuando el robot se apaga.

9. **Programa embebido (Firmware)**

Conjunto de instrucciones grabadas en el microcontrolador, desarrolladas para ejecutar una tarea específica. En este caso, es el algoritmo que permite al Micromouse navegar, mapear el laberinto y buscar la mejor ruta.

10. **Motor DC (de corriente continua)**

Actuador que transforma energía eléctrica en movimiento rotacional continuo. Se conecta a las ruedas del robot para permitir su desplazamiento.

11. **Motor paso a paso (Stepper motor)**

Motor que se mueve en pasos discretos y permite controlar con precisión el ángulo de giro. Es ideal para robots que requieren movimientos controlados, aunque en el Micromouse se prefieren motores DC con encoders por su velocidad.

12. Algoritmo de navegación

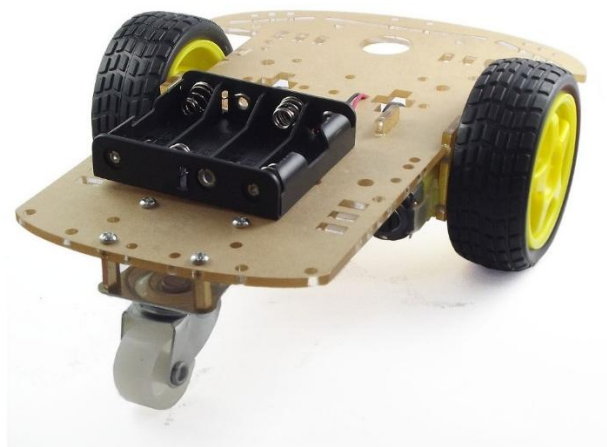
Es un conjunto de reglas y cálculos utilizados por el robot para explorar el laberinto y determinar el camino más corto hacia el objetivo. Ejemplos comunes son: Flood Fill, Wall Following, Depth-First Search (DFS) y Breadth-First Search (BFS).

13. Batería recargable (LiPo, NiMH, etc.)

Fuente de energía portátil que alimenta todos los componentes del Micromouse. Su capacidad y voltaje determinan la autonomía del robot.

14. Chasis robótico

Estructura física del robot que sostiene los componentes mecánicos y electrónicos. Debe ser liviana, compacta y resistente para moverse con agilidad dentro del laberinto.



Chasis Robótico, imagen referencial

Fuente: <https://naylampmechatronics.com/robotica/105-chasis-plataforma-robot-movil-2wd.html>

15. Bus de datos

Canal interno del microcontrolador que transporta información entre sus módulos (memoria, puertos, procesador). Es fundamental para el funcionamiento coordinado del sistema.

16. Registro (Register)

Memoria de muy alta velocidad dentro del microcontrolador, utilizada para guardar temporalmente datos en proceso, como resultados de cálculos o estados del sistema.

17. PWM (Pulse Width Modulation)

Técnica de modulación por ancho de pulso utilizada para controlar la velocidad de los motores ajustando la energía entregada. Es vital para lograr movimientos suaves y precisos.

18. Tiempo real (Real-time)

Capacidad del sistema para responder de forma inmediata a estímulos del entorno. En el Micromouse, esto implica reaccionar a obstáculos o tomar decisiones de navegación en milisegundos.

CARACTERISTICA DEL HARDWARE

Lista de componentes

Componentes	Función	Cantidad
Arduino Nano	Unidad de control principal	1
Driver de motor L298N	Control de motores DC	1
Motor DC TT	Movimiento del robot	2
Protoboard	Ensamblaje temporal de pruebas	1
Chasis 2WD	Soporte para montar componentes	1
Batería 7.4 V	Fuente de energía	1
Sensor ultrasónico HC-SR04	Medición de distancia (frontal y lateral)	2
Cables jumper M-M M-H	conexión	13
Ruedas	Movilidad	3

MOVIMIENTO

1. Chasis

Una base de material consistente que incluya ruedas, rueda loca, soporte para sensores, batería y Arduino.

2. Ruedas

Se utilizará 3 ruedas, 2 controladas por los motores DC y una rueda loca que brindara estabilidad.

- 3. Un motor de corriente continua (DC)** convierte energía eléctrica en movimiento rotatorio. Se alimenta con una tensión constante (por ejemplo, 6V o 12V) y al girar, puede mover ruedas, engranajes, hélices, etc. Es el motor más usado en robótica por su simplicidad y bajo costo.

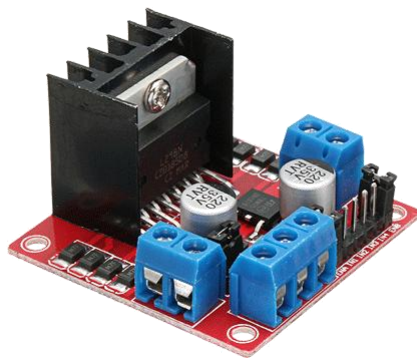
característica	Recomendación
voltaje	6V a 12V
Corriente	<1A (por motor)
RPM	100-300 RPM
Tipo	Con caja reductora
Torque	Alto (para cargar sensores y baterías)



4. Driver L298N (controlara dirección y velocidad)

Es un driver de doble puente H que permite controlar dos motores de corriente continua (DC) de forma independiente, tanto en sentido de giro como en velocidad, mediante señales digitales enviadas desde un microcontrolador como el Arduino Nano

Características	Detalle
Corriente por canal	Hasta 2A por canal (pico)
Voltaje de operación	5V – 35V (motores)
Voltaje logico	3.3V - 5V (compatible con arduino)
Tipo de control	IN1, IN2, IN3, IN4
Dimensiones típicas	4.3 cm × 4.3 cm (placa rectangular)



Funcionamiento del L298N

El control del motor se realiza mediante combinaciones de señales digitales en los pines de entrada. Por ejemplo, para el motor A, se utilizan los pines **IN1** e **IN2**:

- **IN1 = HIGH** y **IN2 = LOW** → el motor gira en un sentido (por ejemplo, hacia adelante).
- **IN1 = LOW** y **IN2 = HIGH** → el motor gira en el sentido contrario (por ejemplo, hacia atrás).
- **IN1 = LOW** y **IN2 = LOW** → el motor se detiene.
- **IN1 = HIGH** y **IN2 = HIGH** → también se detiene (estado de freno o "brake").

Este mismo principio se aplica al motor B con los pines **IN3** e **IN4**.

NAVEGACION

5. Sensores ultrasónicos HC-SR04

El HC-SR04 es un sensor ultrasónico de distancia que mide cuánto tarda un pulso de sonido en rebotar contra un objeto.

Características	
Modelo	HC-SR04
Rango de medición	~2cm a 400cm (4metros)
Precisión	±3mm
Voltaje de operación	5V DC
Salidas	Señales digitales de pulso



Funcionamiento del HC-SR04

Primero emite un pulso ultrasónico por un transductor, el sonido rebota en un objeto y regresa al sensor, midiendo el tiempo de ida y vuelta del sonido se puede calcular la distancia.

CONTROL

6. Arduino Nano

El **Arduino Nano** es una **placa de desarrollo** basada en el microcontrolador **ATmega328P**. Tiene las mismas capacidades que un **Arduino Uno**, pero en un formato mucho más compacto, lo que lo hace perfecto para proyectos donde el **espacio y el peso son críticos**, como los robots móviles.



Características técnicas

Características	Arduino Nano
Tamaño	Muy pequeño (18 mm × 45 mm)
Microcontrolador	ATmega 328P
Velocidad del reloj	16 MHz
Memoria Flash	32 KB (para tu código)
SRAM	2 KB (variables en tiempo real)
EEPROM	1 KB (almacenamiento permanente)
Voltaje de operación	5V
Entradas/salidas (I/O)	14 digitales (6 PWM), 8 analógicas
Alimentación	USB mini-B, pin VIN
Conector USB	Mini-USB (más compacto)
Uso ideal	Robots pequeños y proyectos compactos

El Arduino Nano controlara:

- **Los Motores DC** (a través del driver L298N)
- Sensores ultrasónicos (HC-SR04)
- Lógica de navegación (algoritmo del laberinto)

ALIMENTACION

Se necesitará alimentar los siguientes componentes:

- Arduino Nano
- Motores DC
- Driver L9110S
- Sensores (HC-SR04, IR)

La opción más recomendada será:

Pack de baterías 18650 (recargables de litio)



Usaremos 2 celdas de 3.7V así que en total 7.4V, son ligeras y comunes en proyectos de robótica con una duración de entre 1 y 2 horas.

La conexión será al pin VIN del Arduino, también se conecta al driver L9110S para alimentar los motores.

Para recargar las baterías podemos usar cargadores específicos balanceados (por ejemplo: IMAX B6)

Ensamblaje y Conexiones

Para construir el robot móvil autónomo, se integraron varios componentes electrónicos y mecánicos sobre un chasis 2WD. El objeto principal del ensamblaje fue lograr un sistema capaz de desplazarse de manera autónoma dentro de un entorno controlado, reaccionado ante obstáculos mediante sensores ultrasónicos.

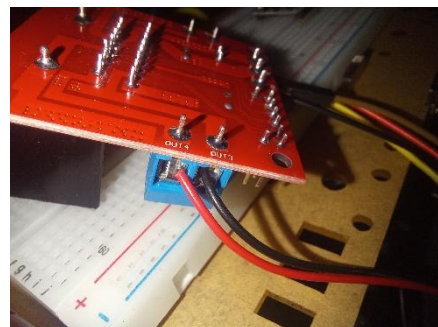
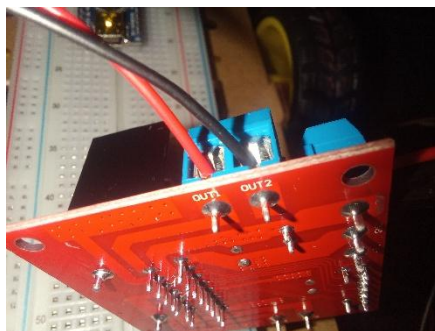
A continuación, se detalla cómo y porque se realizó cada conexión:

Conexión de Motores DC al Driver L298N

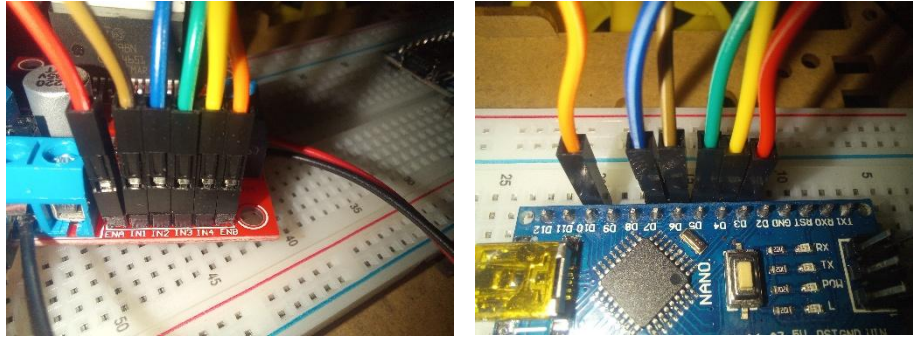
Se utilizaron dos motores DC que se conectaron directamente al módulo L298N, el cual actúa como un puente H doble. Este driver permite controlar tanto la dirección como la velocidad de los motores a través de señales digitales y PWM (modulación por ancho de pulso).

Las conexiones se realizaron de la siguiente manera:

- Motor A(izquierdo): conectado a las **OUT1** y **OUT2** del L298N.
- Motor B(derecho): conectado a las salidas **OUT3** y **OUT4** del L298N.



- Las entradas de control del L298N se conectaron al Arduino nano:
 - **IN1** → Pin 6(marrón)
 - **IN2** → Pin 7(azul)
 - **IN3** → Pin 5(verde)
 - **IN4** → Pin 4(amarillo)



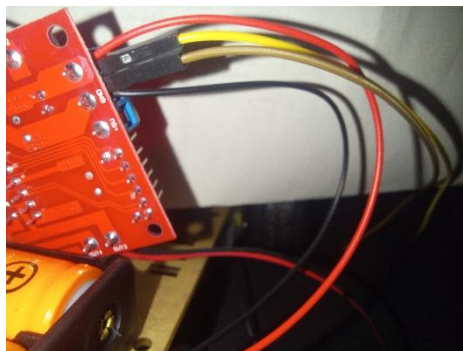
- Las entradas de velocidad (ENA y ENB) se conectaron a pines PWM:
 - **ENA** → Pin 3(rojo)
 - **ENB** → Pin 10(naranja)

Esto permite al Arduino controlar la velocidad y sentido de giro de cada motor individualmente.

Conexión de alimentación

El sistema se alimenta con una batería Li-ion de 7.4V(dos celdas)

- La batería se conecta al terminal de entrada de voltaje del L298N, lo cual alimenta tanto a los motores como al Arduino.
- El pin **VIN** (amarillo) del Arduino Nano se conecta directamente a esta fuente, permitiendo que el regulador interno del Arduino suministre 5V al resto del sistema.
- La conexión común a **GND** (marrón) es esencial para un funcionamiento correcto y debe compartirse entre la batería, el Arduino y el L298N.



Conexión de Sensores ultrasónicos HC-SR04

Se emplearon dos sensores ultrasónicos HC-SR04: uno frontal y uno lateral. Estos sensores permiten medir la distancia a los obstáculos mediante la emisión y recepción de ondas de ultrasonido.

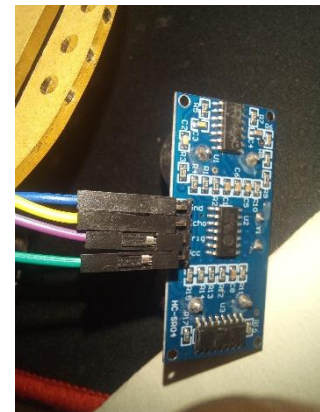
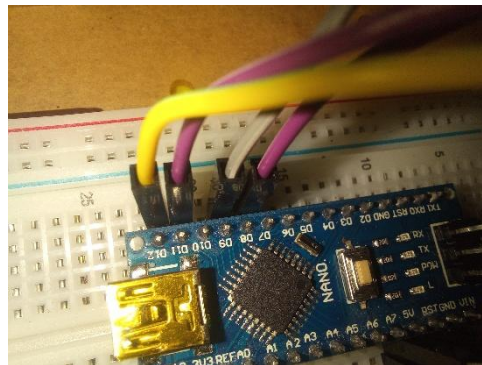
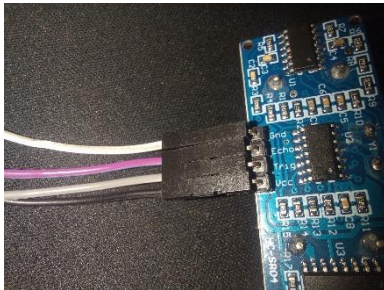
Cada sensor requiere dos conexiones digitales:

- TRIG: envía un pulso ultrasónico
- ECHO: recibe el pulso rebotado para calcular la distancia

Las conexiones se realizaron así:

- Sensor frontal:
 - **TRIG** → Pin 9(plomo)
 - **ECHO** → Pin 8(morado)
 - **GND** → Gnd Arduino(negro)
 - **VCC** → 5V Arduino(blanco)
- Sensor lateral:
 - **TRIG** →Pin 11(morado)
 - **ECHO** →Pin 12(amarillo)
 - **GND** → Gnd Arduino()
 - **VCC** → 5V arduino

Ambos sensores comparten la alimentación de 5V y GND, provista por el Arduino. Se eligieron estos pines digitales por estar disponibles y soportar lectura/escritura con precisión.



Lógica de programación y funcionamiento del código

A continuación, se presenta el código fuente utilizado para el control del robot móvil con sensores ultrasónicos y motores DC, gestionado por una placa Arduino Nano. Se incluye una explicación por bloques:

- **Definición de pines y variables globales**

```
#define TRIG_FRONT 9
#define ECHO_FRONT 8
#define TRIG_LATERAL 11
#define ECHO_LATERAL 12
#define IN1 7
#define IN2 6
#define IN3 5
#define IN4 4
#define ENA 3
#define ENB 10

Long duration_front, duration_lateral;
int distance_front, distance_lateral;
```

Se definen los pines utilizados para los sensores ultrasónicos (frontal y lateral), y para el control de los motores a través del driver L298N. También se declaran variables para almacenar la duración del pulso ultrasónico y las distancias calculadas.

- **Configuración inicial (setup())**

```
void setup() {  
  pinMode(TRIG_FRONT, OUTPUT);  
  pinMode(ECHO_FRONT, INPUT);  
  pinMode(TRIG_LATERAL, INPUT);  
  pinMode(ECHO_LATERAL, INPUT);  
  pinMode(IN1, OUTPUT);  
  pinMode(IN2, OUTPUT);  
  pinMode(IN3, OUTPUT);  
  pinMode(IN4, OUTPUT);  
  pinMode(ENA, OUTPUT);  
  pinMode(ENB, OUTPUT);  
  Serial.begin(9600);  
}
```

Se configuran los pines según su función: salida para los pines de control y entrada para los sensores. También se inicia la comunicación serial para poder visualizar datos si es necesario.

- **Bucle principal (loop())**

```
void loop() {  
  digitalWrite(TRIG_FRONT, LOW);  
  delayMicroseconds(2);  
  digitalWrite(TRIG_FRONT, HIGH);  
  delayMicroseconds(10);  
  digitalWrite(TRIG_FRONT, LOW);  
  duration_front = pulseIn(ECHO_FRONT, HIGH);  
  distance_front = duration_front * 0.034 / 2;  
  
  digitalWrite(TRIG_LATERAL, LOW);  
  delayMicroseconds(2);  
  
  digitalWrite(TRIG_LATERAL, HIGH);  
  delayMicroseconds(10);  
  digitalWrite(TRIG_LATERAL, LOW);  
  duration_lateral = pulseIn(ECHO_LATERAL, HIGH);  
  distance_lateral = duration_lateral * 0.034 / 2;  
  
  if (distance_front > 20) {  
    moveForward();  
  } else if (distance_front <= 20 && distance_lateral > 15)  
  
  turnRight();  
  delay(500);  
} else {  
  turnLeft();  
  delay(500);  
}  
}
```

Este bloque se ejecuta continuamente. Primero, mide las distancias usando los sensores ultrasónicos.

- Si **no hay obstáculo al frente**, el robot avanza.
- Si **hay un obstáculo al frente pero espacio al costado**, gira a la derecha.
- Si **ambos lados están bloqueados**, gira a la izquierda.

- **Función moveForward()**

```
void moveForward() {  
  digitalWrite(IN1, HIGH);  
  digitalWrite(IN2, LOW);  
  digitalWrite(IN3, HIGH);  
  digitalWrite(IN4, LOW);  
  analogWrite(ENA, 150);  
  analogWrite(ENB, 150);  
}
```

Activa los motores para que el robot avance recto. La velocidad se regula con analogWrite.

- **Función turnRight()**

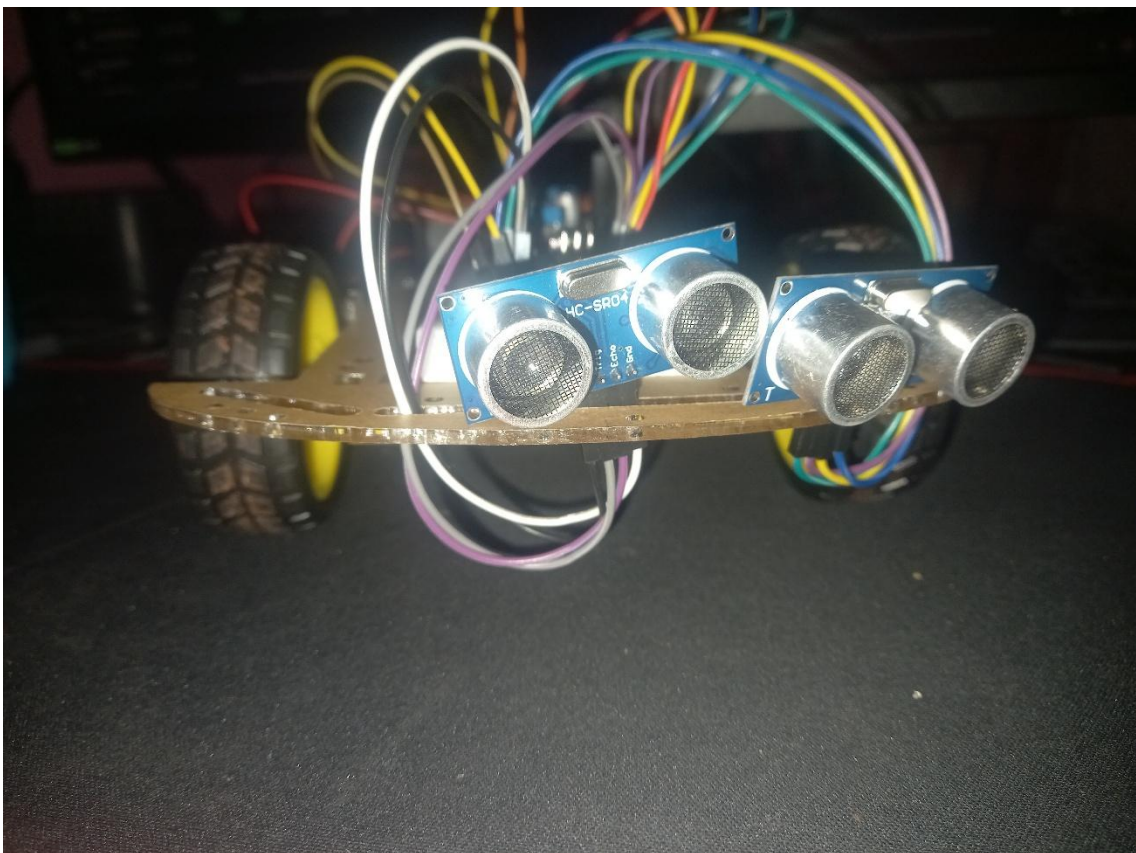
```
void turnRight() {  
  digitalWrite(IN1, LOW);  
  digitalWrite(IN2, HIGH);  
  digitalWrite(IN3, HIGH);  
  digitalWrite(IN4, LOW);  
  analogWrite(ENA, 100);  
  analogWrite(ENB, 100);  
  delay(300);  
  moveForward();  
}
```

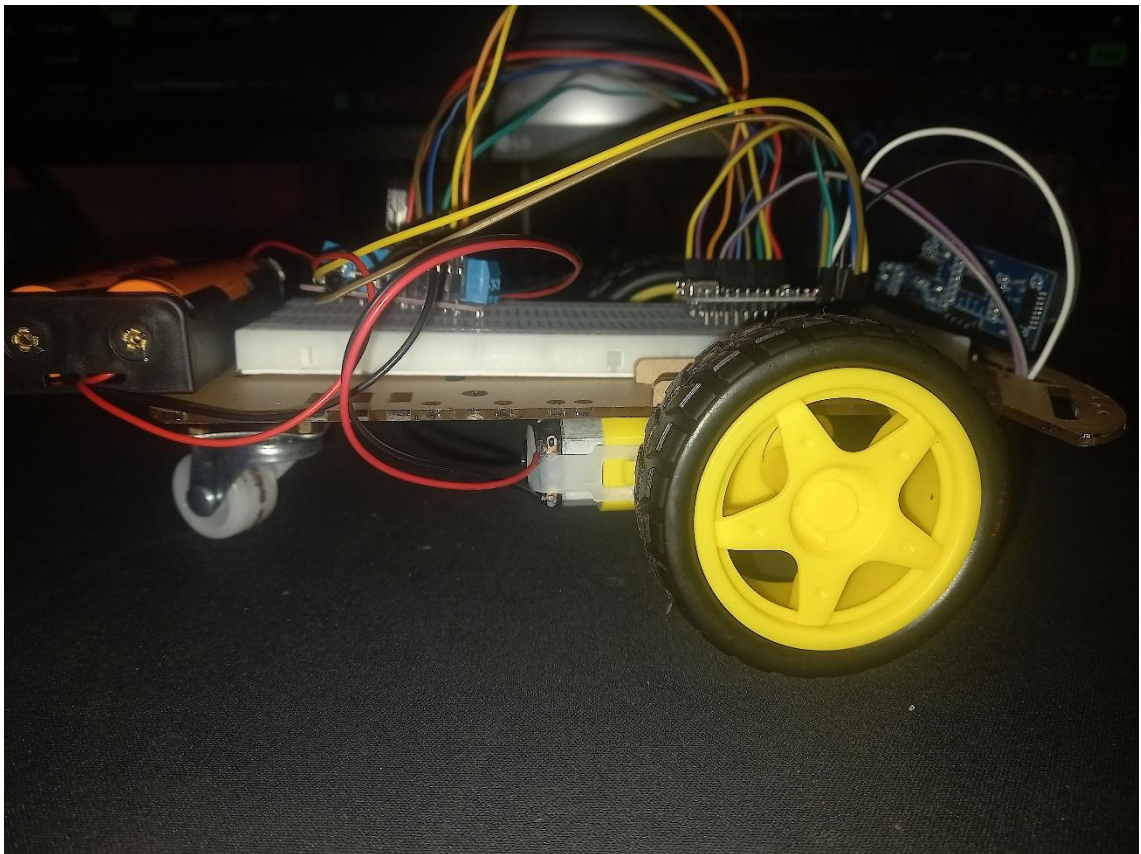
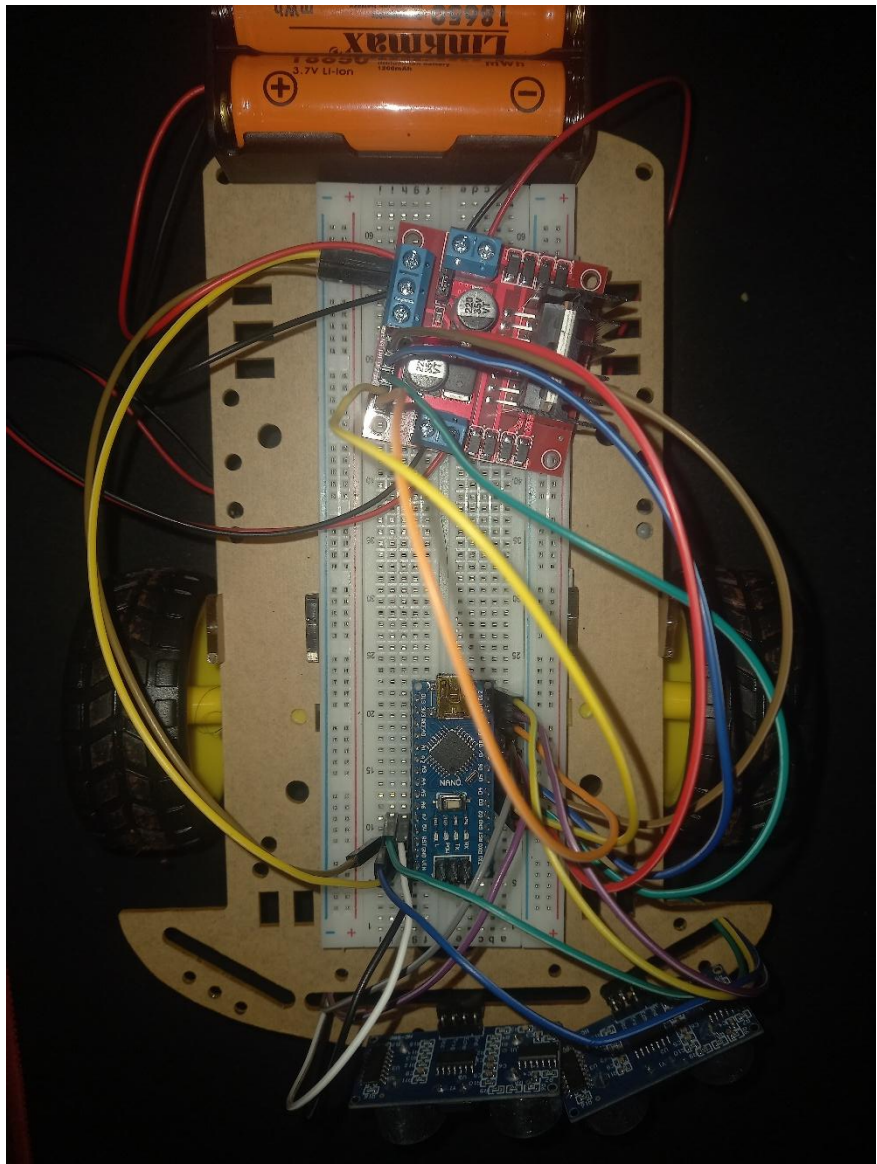
Hace girar el robot hacia la derecha. Un motor avanza y el otro retrocede por un breve tiempo, luego vuelve a avanzar.

- Función turnLeft()

```
void turnLeft() {  
  digitalWrite(IN1, HIGH);  
  digitalWrite(IN2, LOW);  
  digitalWrite(IN3, LOW);  
  digitalWrite(IN4, HIGH);  
  analogWrite(ENA, 100);  
  analogWrite(ENB, 100);  
  delay(300);  
  moveForward();  
}
```

Gira el robot hacia la izquierda. Al igual que la función anterior, se logra girando las ruedas en sentidos opuestos por un momento.





CONCLUSION

- Aplicación práctica del conocimiento

El desarrollo del robot móvil autónomo permitió poner en práctica los conceptos teóricos sobre sensores, actuadores, microcontroladores y control de movimiento, reforzando el aprendizaje de manera integral mediante la implementación física y lógica del sistema.

- Importancia de una buena planificación del hardware

La correcta elección y conexión de los componentes —como el Arduino Nano, el driver L298N, los sensores ultrasónicos y los motores DC— fue fundamental para garantizar el correcto funcionamiento del robot. Cada módulo fue seleccionado no solo por su funcionalidad, sino también por su compatibilidad y eficiencia.

- Comprensión de los principios de navegación autónoma

Al usar sensores ultrasónicos y un algoritmo lógico basado en la distancia medida, se logró que el robot tomara decisiones básicas de navegación, como avanzar, girar o esquivar obstáculos. Esto permitió simular un comportamiento autónomo simple y efectivo.

- Relevancia del control lógico mediante programación

A través del lenguaje de programación de Arduino (basado en C++), se controlaron todos los módulos del robot. El código implementado demuestra cómo una secuencia estructurada de decisiones puede traducirse en movimiento físico, destacando la relación entre software y hardware.

- Versatilidad y ventajas de Arduino en proyectos educativos

Arduino se consolidó como una plataforma accesible y poderosa para el desarrollo de prototipos. Su flexibilidad, bajo costo y amplia comunidad lo convierten en una herramienta ideal para proyectos de automatización y robótica.

- Desarrollo de habilidades técnicas y de resolución de problemas

El ensamblaje del robot y la solución de errores tanto en conexiones como en código fortalecieron habilidades de lógica, análisis y resolución de problemas, que son fundamentales para la carrera de Ciencias de la Computación.

- Potencial de mejora del proyecto

Aunque se logró el objetivo de construir un robot móvil básico, existen múltiples posibilidades de mejora: añadir sensores infrarrojos para seguir líneas, integrar Bluetooth para control remoto, o implementar una batería recargable con módulo de carga seguro.

REFERENCIAS CONSULTADAS

1. Wikipedia. "Micromouse".
<https://en.wikipedia.org/wiki/Micromouse>
2. Arduino Official Documentation. "Arduino Nano".
<https://docs.arduino.cc/hardware/nano>
3. SparkFun. "HC-SR04 Datasheet".
<https://www.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
4. GeeksforGeeks. "Flood Fill Algorithm".
<https://www.geeksforgeeks.org/flood-fill-algorithm/>
5. Wikipedia. "DC Motor".
https://en.wikipedia.org/wiki/DC_motor
6. Wikipedia. "Real-time Computing".
https://en.wikipedia.org/wiki/Real-time_computing
7. Elegoo Starter Kit (manual PDF)
<https://www.elegoo.com/pages/learn>
8. Pololu – Micromotores y ruedas
<https://www.pololu.com/category/22/micro-metal-gearmotors>
9. DFRobot – Alimentación con baterías en robots
https://wiki.dfrobot.com/Battery_Power_for_Robots
10. Paul Scherz – Practical Electronics for Inventors (Libro)
<https://www.amazon.com/Practical-Electronics-Inventors-Fourth-Scherz/dp/1259587541>
11. Datasheet L9110S Motor Driver (Electrodragon)
https://www.electrodragon.com/w/L9110_Driver
12. DroneBot Workshop (YouTube) – Robots con Arduino
<https://www.youtube.com/c/DroneBotWorkshop>